Complex & Intelligent Systems (2025) 11:210
https://doi.org/10.1007/s40747-025-01828-6

ORIGINAL ARTICLE

A heuristic-assisted deep reinforcement learning algorithm for flexible job shop scheduling with transport constraints

Xiaoting Dong[1,2,3] · Guangxi Wan[1,2] · Peng Zeng[1,2]

Received: 21 July 2024 / Accepted: 28 February 2025 / Published online: 17 March 2025
© The Author(s) 2025

Abstract
Automated guided vehicles (AGVs) are widely used for transportation in flexible job shop (FJS) systems, and their transportation task scheduling has the same substantial impact on production efficiency as machine scheduling does. However, traditional FJS scheduling methods often prioritize job sequencing and machine selection while ignoring the impact of AGV transportation, resulting in suboptimal scheduling solutions and even difficulties in implementation. To address this issue, this paper formulates a cooperative scheduling model by introducing the AGV scheduling problem into the classical FJS scheduling problem, abbreviated as the FJS-AGV problem, with the objective of minimizing the makespan. With respect to the FJS-AGV problem, a heuristic-assisted deep Q-network (HA-DQN) algorithm is proposed, which leverages heuristic rules to enable the decision agent to perform multiple actions at each decision point, which includes determining the responses to the following questions: Which operation should be processed next? On which machine? By which AGV? This decision mechanism enables the agent to make more informed decisions, leading to improved performance and resource allocation in the FJS-AGV system. The practicability of the proposed FJS-AGV model and the efficiency of the HA-DQN algorithm in solving the FJS-AGV problem are verified through various international benchmarks. Specifically, when solving instances in a large benchmark, the HA-DQN algorithm yields a significant 12.63% reduction in makespan compared with that when traditional heuristics are employed.

Keywords Flexible manufacturing system · Cooperative scheduling of machines and AGVs · Markov decision process · Heuristic-assisted DQN algorithm

Introduction

The FJS scheduling problem is an intractable NP-hard problem that exists widely in various domains, such as semiconductor manufacturing, steel refining, and automotive production. In the FJS system, each job requires multiple operations that must be transferred to different machines for processing. To ensure efficient transportation of these operations, many enterprises select AGVs as transport resources to automatically transport jobs [1]. Machines and AGVs serve as the production implementation entities in FJS systems, and their behaviors have spatiotemporal interactions. The machine selection for operation processing and the operation process sequence directly affect the AGV transportation task assignment and moving routes. In contrast, the transportation stages for jobs by AGVs heavily restrict the start time of subsequent operations and thereby affect the order completion time. The machine processing tasks and AGV transportation tasks are highly coupled in flexible job shops, and decoupling or hierarchical optimization often leads to suboptimal solutions. Furthermore, the load and no-load behavior of the AGVs alternate during the production process, including some necessary idle waiting phases, which has a complex cascading effect on the calculation of the order completion

✉ Guangxi Wan
wanguangxi@sia.cn

✉ Peng Zeng
zp@sia.cn

Xiaoting Dong
dongxiaoting@sia.cn

1 State Key Laboratory of Robotics, Shenyang Institute of Automation, Chinese Academy of Sciences, Shenyang 110016, China

2 Key Laboratory of Networked Control Systems, Shenyang Institute of Automation, Chinese Academy of Sciences, Shenyang 110016, China

3 University of Chinese Academy of Sciences, Beijing 100049, China

time. Therefore, it is necessary to address the machine and AGV scheduling problem simultaneously in FJS systems.

However, traditional FJS scheduling problem approaches often assume that the transport tools are always ready and available at any time, so the scheduling process often ignores the material transfer time or sets it as a fixed value [2]. This assumption is true for flexible job shop systems that can ensure unlimited transport capacity, but in practical manufacturing systems, transportation resources are often limited. Furthermore, traditional approaches employed to address the FJS scheduling problem, such as dispatching rules and metaheuristics, face challenges in terms of time efficiency and guarantee high-quality solutions [3]. These approaches either suffer from slow processing speeds or struggle to consistently achieve optimal solutions that fulfill different objectives. In recent years, deep reinforcement learning (DRL) algorithms [4] have enabled significant advancements owing to their exceptional abilities in self-learning and self-optimization, which has allowed them to solve complex decision-making problems swiftly and accurately and has resulted in their extensive utilization across various job shop scheduling problems [5, 6].

Inspired by the aforementioned cooperative scheduling requirement and the potential of DRL algorithms, we integrate the AGV scheduling problem and classical FJS scheduling problem as a unified problem and model it as a Markov decision process. Furthermore, a hybrid optimization algorithm based on DRL is developed to effectively solve the cooperative scheduling problem. The major contributions of this research can be summarized as follows:

1. We formulate the FJS-AGV problem as a Markov Decision Process (MDP), independent of scale, with an integrated decision-making approach which combines the operation selection, machine selection and AGV selection decisions as one decision at each decision-making time.
2. A heuristic-assisted deep Q-network (HA-DQN) algorithm is proposed to solve the FJS-AGV problem, which utilizes heuristic scheduling rules to select machines and AGVs for the operations determined by the DQN at each decision point. This method effectively reduces the dimensionality of the DQN action space and speeds up the learning process for decision-making strategies.
3. The effectiveness and generalization capacity of the proposed HA-DQN algorithm are validated on five benchmark datasets. Compared to traditional (meta-)heuristic algorithms, HA-DQN shows marked superiority in both solution quality and computational speed. Furthermore, when benchmarked against DRL methods, HA-DQN also demonstrates outstanding generalization performance.

The remaining sections of this article are organized as follows. Section Related work summarizes the relevant literature on this topic. Section Problem description formulates the mathematical model of the FJS-AGV problem. Section Methodology architecture for the FJS-AGV problem outlines the methodology used in this study. Section Experiments and results presents the experiments conducted and the corresponding results. Finally, Sect. Conclusion concludes this paper by summarizing the key findings.

Related work

This paper aims to solve the FJS-AGV problem via DRL-based algorithm. Therefore, this section provides a summary of the research status in two aspects: the research status of the FJS-AGV problem and the research status of using DRL algorithms to solve job shop scheduling problems.

Existing works on the FJS-AGV problem

The FJS-AGV problem is an extension of the FJS scheduling problem that incorporates transport resources and tasks. The practical urgency of cooperative scheduling and its difficulty have attracted the attention of several researchers. The first published work dates back to 2009, when Caumond et al. [7] formulated a mixed integer linear programming (MILP) model for an FJS with a single AGV, considering the restriction of buffer capacities. Novas et al. [8] and Ham et al. [9] proposed a constraint programming model for an FJS scheduling problem with AGVs. The former focused on the comprehensive scheduling of limited workshop resources, including equipment resources such as machines and AGVs, as well as space resources such as buffer capacities. The latter paid more attention to the logic relationship analysis of AGV behaviors. Homayouni and Fontes [10] pioneered the MILP model for FJS with limited AGVs, addressing small-scale instances through a local search-based heuristic. Kumar et al. [11] employed a differential evolution algorithm to solve the FJS-AGV problem, aiming to minimize the makespan. Nouri [12] introduced a hybrid metaheuristic based on a clustered holonic multiagent model to expedite makespan estimation in the FJS-AGV problem. Espinouse et al. [13] demonstrated that the FJS-AGV problem, even with a single machine, one depot, and one AGV, is strongly NP-hard. These early studies explored FJS-AGV problems from various dimensions, such as AGV capability, machine constraints, and optimization objectives, establishing corresponding mathematical models that laid the groundwork for future research on FJS-AGV problems.

With increasing production scale and product diversification, FJS-AGV problem complexity has increased, mainly due to increased tasks, diversified constraints, and conflict-

ing optimization objectives. This complexity challenges the speed and accuracy of precise algorithms. To address this, researchers have started exploring heuristic and hybrid algorithms for the FJS-AGV problem. Dang et al. [14] proposed a hybrid heuristic algorithm that combines a genetic algorithm and tabu search to solve the problem with makespan minimization. By incorporating an energy consumption constraint, Tan et al. [15] further expanded the MILP model. Cheng et al. [16] designed an adaptive ensemble neural network to minimize makespan in FJS-AGV problem. Yan et al. [17] explored the application of digital twin technology in the FJS-AGV problem, employing an enhanced genetic algorithm and introducing a novel Entity-JavaScript Object Notation method to visualize solutions in the digital twin system. Chen et al. [18] hybridized the particle swarm algorithm with a competitive learning mechanism and a random restart mechanism to solve the machine and AGV cooperative scheduling problem in a flexible flow shop. In addition, Yuan et al. [19]), Wen et al. [20] and He et al. [21] studied the multiobjective FJS scheduling problem with AGVs and adopted the NSGA-II algorithm and multiobjective evolutionary algorithm (EMOEA) to solve it. Although heuristic algorithms were once considered optimal solvers for FJS-AGV problems, they are highly parameter dependent, prone to local minima, and their solving time increases exponentially with increasing problem complexity.

Recently, Pan et al. [22] introduced an innovative bi-learning evolutionary algorithm tailored to address the joint scheduling challenge of production and AGVs in distributed flexible job shops. By integrating a probability-based parameter learning mechanism with the evolutionary algorithm (EA), they significantly bolstered the EA's capacity to explore optimal solutions. In a separate study [1], the same researchers employed a learning-based multipopulation evolutionary algorithm to address the FJS-AGV problem. They devised a multipopulation evolutionary mechanism and incorporated a reinforcement learning algorithm for proactive selection of EA evolution parameters, thereby significantly enhancing the diversity and evolutionary vitality of the EA population. Furthermore, Zhang et al. [23] proposed a memetic algorithm to solve the FJS-AGV problem, where a DQN-based model is designed to guide the evolutionary algorithm to consistently select optimal operators. These aforementioned contributions have markedly improved the algorithm's efficiency in addressing the FJS-AGV problem by embedding RL-grounded parameter active learning mechanisms within heuristic methodologies. This progress has sparked our interest in exploring further potential applications of RL algorithms in solving the FJS-AGV problems.

Although the aforementioned research has enhanced the efficiency of heuristic algorithms in solving the FJS-AGV problem, heuristic algorithms are time-consuming in nature, especially for possible online scheduling. Instead, the proposed HA-DQN is a heuristic-enhanced learning approach that possesses a certainty of generalization in solving different instances. Within the HA-DQN framework, the agent determines the optimal processing operations and the heuristic dispatch rules select suitable machines and AGVs, accordingly. The HA-DQN method is characterized with the ability of the exploration and exploitation of DRL, and it can effectively reduce the exploration space dimension by utilizing heuristic rules. Furthermore, once the HA-DQN model is trained, it can be directly transferred and applied to similar problems without retraining, thus improving the efficiency of solving scheduling problems.

Solving job shop scheduling problems via DRL

DRL algorithms leverage the robust perception capabilities of deep learning to handle intricate and high-dimensional environmental characteristics and take advantage of the concept of reinforcement learning to interact with the environment to complete the decision-making process. Its successful applications in the areas of Go game (Silver et al. [24]; Vinyals et al. [25]) greatly inspired researchers to employ it to solve various shop-floor scheduling problems. Initially, researchers focused primarily on solving the job shop scheduling problem (JSSP) via DRL and later shifted to solving the FJSP.

Lin et al. [26] established a smart factory framework on the basis of edge computing and designed a multiclass DQN algorithm to handle the JSSP within this framework. Palombarini and Martinez [27] used the DQN algorithm to store the rescheduling knowledge and learn rescheduling strategies from high-dimensional inputs directly. Liu et al. [28] proposed a parallel algorithm that integrates an asynchronous update mechanism with a deep deterministic policy gradient to solve the JSP. Han et al. [29] presented a JSP model that is based on the dueling DQN; the information concerning the processing time and machine utilization rate is included in the model, and a convolutional neural network is used to approximate the state-action values. Unfortunately, this model only performs well on small-scale problems. Libing Wang [30] and Goytom et al. [31] utilized the proximal policy optimization (PPO) algorithm to address the standard JSP, verifying that the PPO algorithm experts can effectively manage the challenge of expanding state and action spaces resulting from the increasing scale of the problem, thus mitigating the potential "dimension disaster". Additionally, the PPO algorithm was employed to address the dynamic JSSP by Ming Zhang et al. [32] and Xinquan Wu et al. [33]. Zhang's work focused on handling dynamic events such as unexpected machine failures, whereas Wu's work focused more on the challenge of an uncertain processing time. Additionally, Sun et al. [34] designed a 2D-PPO algorithm with a two-dimensional action

space to accelerate the joint scheduling decisions of machines and AGVs in a job shop. Liu et al. [35] proposed a deep multiagent reinforcement learning framework based on the double-DQN algorithm, and a centralized training and decentralized execution scheme and parameter-sharing technique were introduced to address the nonstationary problem in multiagent reinforcement learning.

Shu Luo et al. [36] solved the FJSP with new job insertion via the DQN algorithm, which consists of seven full layers. Another study by Shu Luo et al. [37] introduced a hierarchical multiagent proximal policy optimization algorithm to solve the dynamic multiobjective FJSP. The higher-level controller determines the temporary objectives to be optimized; two machine agents are lower actuators, one is responsible for operation selection and the other is responsible for machine assignment, thus achieving the temporary objective at each rescheduling point. Kun Lei et al. [38] introduced an end-to-end deep reinforcement framework that leverages GNNs to autonomously learn a policy to solve the FJSP. To accomplish this, they proposed a multipointer graph network architecture and a multi-proximal policy optimization algorithm to train two subpolicies: an operation action policy and a machine action policy. The purpose of these subpolicies is to allocate job operations appropriately to the respective machines within the scheduling problem. Zhang et al. [39] proposed a new FJSP model on the basis of deep reinforcement learning with multi-agent graphs to integrate DRL and MAG and bestow agents with the ability to cooperate. Li et al. [40] proposed a coevolutionary framework to solve the distributed heterogeneous flexible job shop scheduling problem (DHFJS), in which a parasitic behavior-based NSGA-II algorithm is used to generate and search for scheduling solutions under the guidance of the DQN algorithm. Zhang et al. [41] and Zhao et al. [42] both used the PPO algorithm to solve the FJSP with uncertain events. The former studies address the problem of variable processing time, whereas the latter study handles the issues of random job arrivals. Yuan et al. [43] devised a lightweight multilayer perceptron as the state embedding network when PPO was used to solve the FJSP, which achieved good performance in reducing computational complexity.

Discussion of the research gaps

In Sect. Related work, a systematic review of the related research on FJS-AGV and DRL is conducted. Nevertheless, some research gaps still exist, which are summarized as follows:

1. Robustness and scalability challenges: The current mainstream methods for solving the FJS-AGV problem are exact methods and heuristics, which are time-consuming and rely on the specific characteristics of the problem itself. In addition, heuristic algorithms focus mainly on solving small-scale FJS-AGV problems. Therefore, there is a need for the development of more robust and effective algorithms that are specifically tailored to addressing the challenges of large-scale FJS-AGV problems.
2. Lack of integration between FJS-AGV and DRL: Although there has been much research on the use of DRL algorithms to solve job shop scheduling problems, most of these studies have focused on traditional job shop scheduling problems. Therefore, exploring the application of DRL algorithms to solve FJS-AGV problems is still challenging.

Problem description

A flexible job shop, depicted in Fig. 1, often consists of a raw material warehouse (abbreviated as RW), intelligent processing machines, AGVs, a production warehouse (abbreviated as PW) and intelligent communication and controlling equipment. After order activation, the AGVs select jobs from the RWs, transport them to designated machines for processing according to process constraints, and finally transport the finished products to the PW for storage. The FJS-AGV problem can be described as follows. A set $J = \{J_1, J_2, \ldots, J_n\}$ of n jobs, a set $M = \{M_1, M_2, \ldots, M_m\}$ of m machines and a set $V = \{V_1, V, \ldots, V_v\}$ of v AGVs are given. Each job J_i comprises a specific sequence of n_i consecutive operations $O_i = \{O_{i1}, O_{i2}, \ldots, O_{in_i}\}$ with precedence constraints. An operation O_{ij} can be processed on a subset of eligible machines $M_{ij} \subseteq M$. AGVs are responsible for transporting the jobs between machines, and each AGV can transport any type of workpiece, with the capacity to hold only one at a time. The optimization goal of FJS-AGVs is to (1) assign operations to compatible machines, (2) determine a process sequence of operations on each machine, (3) assign operations to appropriate AGVs, and (4) determine a transport sequence of operations on each AGV to minimize the makespan $C_{\max}$ described by Eq. 1, where E_{in_i} denotes the completion time of job J_i. To express the FJS-AGV problem more precisely, some notations and variables are defined and shown in Table 1 and Table 2.

$$C_{max} = \max\{E_{in_i}\}, i \in \{1, \ldots, n\} \tag{1}$$

Constraints

$$\sum_{l=1}^{m} Y_{i,e,l} = 1, \forall i \in \{1, 2, \ldots, n\}, \forall e \in \{1, 2, \ldots, n_i\} \tag{2}$$

$$\sum_{r=1}^{v} YT_{i,e,r} = 1, \forall i \in \{1, 2, \ldots, n\}, \forall e \in \{1, 2, \ldots, n_i\} \tag{3}$$

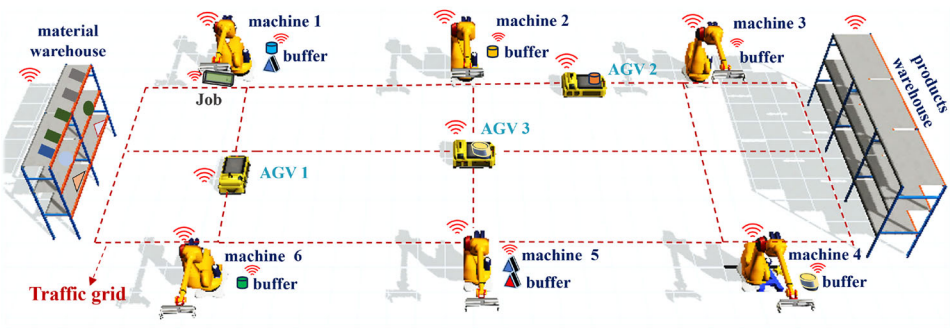


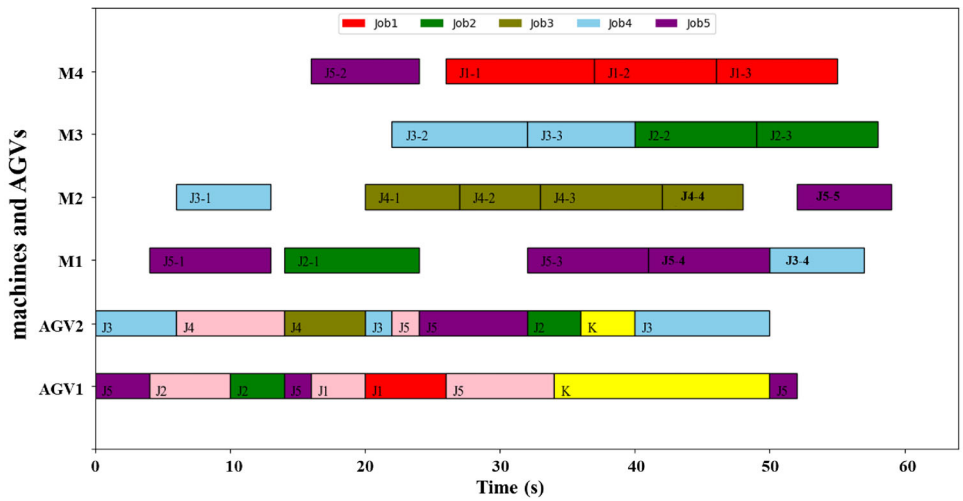
Fig. 1 Flexible job shop layout diagram

Table 1 Notations in FJS-AGV

No	Notation	Description
1	n	Number of jobs
2	m	Number of machines
3	v	Number of AGVs
4	i, h	Index of jobs, $i, h = 1, 2, \ldots, n$
5	k, l	Index of machines, $k, l = 1, 2, \ldots, m$
6	e, f	Index of operations, $e, f = 1, 2, \ldots, n_i$
7	r, s	Index of AGVs, $r, s = 1, 2, \ldots, v$
8	n_i	Number of operations of job i
9	n_{max}	The total number of operations of all jobs
10	$O_{i,e}$	The eth operation of job i
11	$S_{i,e,l}$	The start time of the eth operation of job i on machine l
12	$P_{i,e,l}$	The processing time of the eth operation of job i on machine l
13	$E_{i,e,l}$	The end time of the eth operation of job i on machine l
14	$ST_{i,e,r}$	The transport start time of the eth operation of job i on AGV r
15	$UT_{i,e,r}$	The no-load transport time of the eth operation of job i on AGV r
16	$LT_{i,e,r}$	The load transport time of the eth operation of job i on AGV r
17	A	The action space of the HA-DQN algorithm

Table 2 Notations of the decision variables

No	Notation	Description
1	$Y_{i,e,l}$	1: the eth operation of job i is assigned to machine l
		0: the eth operation of job i is assigned to machine l
2	$YT_{i,e,r}$	1: the transport task of the eth operation of job i is assigned to AGV r
		0: the transport task of the eth operation of job i is not assigned to AGV r
3	$Y_{i,e,l}(t)$	1: at time t, the eth operation of job i has been processed on machine l
		0: at time t, the eth operation of job i has not been processed on machine l
4	$YT_{i,e,r}(t)$	1: at time t, the eth operation of job i has been transported by AGV r
		0: at time t, the eth operation of job i has not been transported by AGV r
5	$Y^{l}_{i,e\rightarrow h,f}$	1: the eth operation of job i is processed before the fth operation of job h on machine l
		0: the eth operation of job i is processed after the fth operation of job h on machine l
6	$YT^{r}_{i,e\rightarrow h,f}$	1: the eth operation of job i is transported before the fth operation of job h by AGV r
		0: the eth operation of job i is transported before the fth operation of job h by AGV r



Notes: for AGVs, red rectangles represent the no-load action; yellow rectangles represent the idle and standby state; all others represent the load action.

Fig. 2 Gantt chart of the example problem fjsp3 obtained by HA-DQN

Equations 2 and 3 are the job constraints, which means that one operation can be processed on only one machine and transported by one AGV.

$$ET_{i,e,r} = ST_{i,e,r} + UT_{i,e,r} + LT_{i,e,r} \tag{4}$$

$$S_{i,e,l} = max\{ET_{i,e,r}, MR_l\} \tag{5}$$

$$e_{i,e,l} = S_{i,e,l} + P_{i,e,l} \tag{6}$$

Equations 4, 5 and 6 are the time constraints. The earliest transport end time of an operation can be calculated via Eq. 4. Equation 5 indicates the earliest processing start time of the eth operation of job i on machine l. Equation 6 reflects the relationship between the processing start time, processing time and processing end time of an operation.

$$\sum_{i,h=1}^{n} \sum_{e,f=1}^{n_i} Y^l_{i,e\to h,f} * Y_{i,e,l}(t) <\leq 1, l \in 1, 2, \ldots, m \tag{7}$$

$$\sum_{i,h=1}^{n} \sum_{e,f=1}^{n_i} YT^r_{i,e\to h,f} * YT_{i,e,r}(t) <\leq 1, r \in 1, 2, \ldots, v \tag{8}$$

Equation 7 is the machine constraint, indicating that one machine can process only one job at any time t. Equation 8 is the AGV constraint, indicating that one AGV can transport only one job at any time t.

Assumptions

Some assumptions that need to be satisfied to solve the FJS-AGV problem are as follows:

1. All jobs, machines and AGVs are viable at time zero.
2. All jobs and AGVs were initially located in the RW.
3. Each machine has a buffer with infinite capacity. When a job is transported to a machine, it joins the buffer queue and awaits processing. Similarly, after processing is completed, the job resides in the buffer area until an AGV is ready to transport it to the next processing machine.
4. Once a transportation task has been completed, the AGV docks at its current location and awaits triggering by the next transportation task.
5. If the pre- and postoperations of the same job are performed on the same machine, there is no need to mobilize the AGV for transportation.

An example of the FJS-AGV problem

To provide an intuitive illustration of the FJS-AGV problem, an instance, named fjsp3, sourced from the benchmark by Kumar et al. [11] is given. The problem fjsp3 includes 5 jobs $J = \{J_1, J_2, J_3, J_4, J_5\}$ with $\{3, 3, 4, 4, 5\}$ operations. Each operation has three alternative processing machines with different capabilities, and 2 AGVs are available for job transportation. Figure 2 shows a solution of this FJS-AGV problem in Gantt chart form.

The operations processing sequence and corresponding processing machines and transport AGVs are $O_{5,1}(M_1, V_1)$, $O_{3,1}(M_2, V_2)$, $O_{2,1}(M_1, V_1)$, $O_{4,1}(M_2, V_2)$, $O_{5,2}(M_4, V_1)$, $O_{1,1}(M_4, V_1)$, $O_{3,2}(M_3, V_3)$, $O_{3,3}(M_3, \phi)$, $O_{4,2}(M_2, \phi)$, $O_{4,3}(M_2, \phi)$, $O_{5,3}(M_1, V_2)$, $O_{4,4}(M_2, \phi)$, $O_{1,2}(M_4, \phi)$, $O_{2,2}(M_3, V_2)$, $O_{5,4}(M_1, \phi)$, $O_{2,3}(M_3, \phi)$, $O_{3,4}(M_1, \phi)$, $O_{1,3}(M_4, \phi)$, and $O_{5,5}(M_2, V_1)$. As shown in Fig. 2, the FJS-AGV problem consists of machine scheduling and AGV scheduling. The scheduling process involves four types of AGV activities:

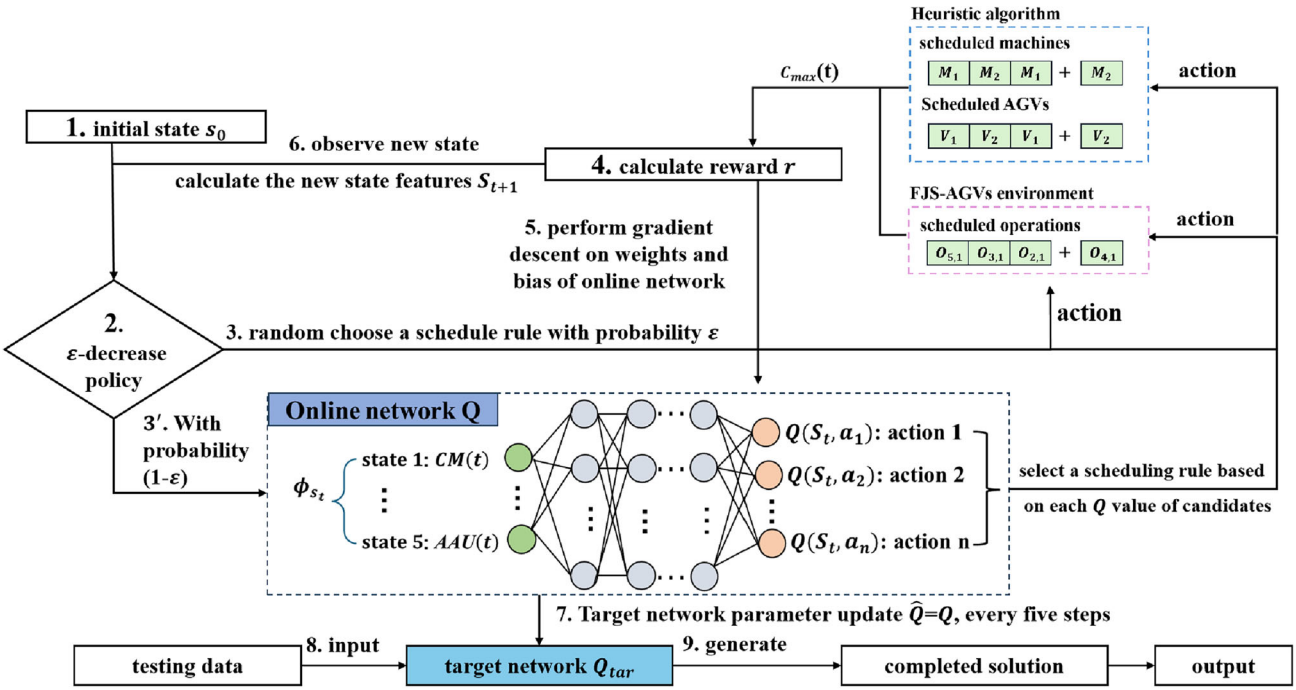


Fig. 3 Algorithm framework for the FJS-AGV problem

1. *Loading jobs on the RW as the first transport task:* the first operation of each job must be loaded on the RW. For example, $O_{5,1}(M_1, V_1)$, V_1 loads job J_5 on the RW and then transports it to M_1. The time cost $trans_{O_{5,1}}$ of transporting $O_{5,1}$ only comprises one part: the transport time from the RW to M_1, $t_{R\to M_1} = 4$; $t_{O_{5,1}} = t_{R\to M_1}$.
2. *Loading jobs on the RW but not as the first transport task:* such as $O_{2,1}(M_1, V_1)$, the transport time of $O_{2,1}$ comprises two parts: (1) V_1 leaves the current position M_1 and returns to the RW to load J_2 (no-load movement, $t_{M_1\to RW} = 6$); and (2) V_1 takes J_2 to M_1 (load movement, $t_{M_1\to RW} = 4$); and the time cost $t_{O_{2,1}} = t_{M_1\to R} + t_{R\to M_1}$.
3. *Transporting jobs for intermediate operations processing:* such as $O_{5,3}(M_1, V_2)$, V_2, leave current position M_3, heading to the processing machine M_4 of $O_{5,2}$ to load J_5 (no-load movement, $t_{M_3\to M_4} = 2$. Note that if the AGV arrives earlier than the processing end time $E_{5,2,4}$, it will result in a waiting time $wt_{5,3} \neq 0$ and then take J_5 to M_1 for processing (load movement, $t_{M_4\to M_1} = 8$); the time cost $t_{O_{5,3}} = t_{M_3\to M_4} + wt_{5,2} + t_{M_4\to M_1}$.
4. *There is no need for an AGV:* for example, when $O_{4,2}(M_2, \phi)$ is processed on the same machine as $O_{4,1}(M_2, V_2)$, no AGV is needed to transport $O_{4,2}$; thus, $t_{O_{4,2}} = 0$.

Methodology architecture for the FJS-AGV problem

In this section, we propose the HA-DQN algorithm, wherein heuristic scheduling rules are employed to select machines and AGVs while the DQN focuses solely on operations selection, to solve the FJS-AGV problem. Figure 3 shows a visualization of the HA-DQN framework.

Definition of the Markov decision process

Modeling the FJS-AGV problem as a MDP is the precondition of implementing the DRL algorithm. An MDP model mainly comprises a 5-tuple (S, A, P, R, γ). S represents the state of the environment interacting with the decision agent. A is the action set. P is the state transition probability. R refers to the immediate reward when executing an action. γ is the reward discount factor.

State features

The agent perceives the environment through the state features and subsequently takes an action to interact with the environment, step by step, until reaching the target state. The state features need to capture the environment characteristics accurately while avoiding redundant computations. Here, five state features are designed according to the characteristics of FJS-AGV and the optimization objective.

Table 3 Notations in FJS-AGV

No	Notation	Description
1	SJPT	The agent selects the job with the shortest processing time
2	LJPT	The agent selects the job with the longest processing time
3	SOPT	The agent selects the operation with shortest processing time
4	LOPT	The agent selects the operation with longest processing time
5	SJRPT	The agent selects the job with smallest rest processing time
6	LJRPT	The agent selects the job with longest rest processing time
7	EOST	The agent selects the operation with the earliest start time for processing
8	SMOPS	The agent selects the operation whose processing machine is the same with its predecessor
9	SOTT	The agent selects the operation with the shortest transport time

1. Current makepan, $CM(t)$

$$CM(t) = \max\{E_{i,e,l}(t)\} \tag{9}$$

In Eq. 9, $E_{i,e,l}(t)$ indicates the completion time of currently scheduled operations at decision point t.

2. Current total process time, $CPT(t)$

$$CPT(t) = \sum_{i=1}^{n}\sum_{e=1}^{n_i}\sum_{l=1}^{m} P_{i,e,l} \times Y_{i,e,l}(t) \tag{10}$$

The $CPT(t)$ in Eq. 10 refers to the accumulated processing time of all workpieces that have been completed until the current decision point t.

3. Average completion percentage, $ACP(t)$

$$CP(t) = (CP_1(t), \ldots, CP_i(t), \ldots, CP_n(t)) \tag{11}$$

$$CP_i(t) = cn_i(t)/n_i \tag{12}$$

$$ACP(t) = 1/n \sum_{i=1}^{n} CP_i(t) \tag{13}$$

In Eq. 11, $CP_i(t)$ represents the completion percentage of job J_i. In Eq. 12, $cn_i(t)$ indicates the number of completed operations of job J_i at decision point t.

4. Average machine utilization, $AMU(t)$

$$MU(t) = (MU_1(t), \ldots, MU_l(t), \ldots, MU_m(t)) \tag{14}$$

$$MU_l(t) = \frac{(\sum_{i=1}^{n}\sum_{e=1}^{n_i} P_{i,e,l} \times Y_{i,e,l}(t))}{CPT(t)} \tag{15}$$

$$AMU(t) = \frac{1}{m}\sum_{l=1}^{m} MU_l(t) \tag{16}$$

In Eq. 14, $MU_l(t)$ represents the lth machine utilization at decision point t, which is equal to the ratio of the sum of the processing time for all operations processed on machine m_l to the total processing time $CPT(t)$, as shown in Eq. 15.

5. Average AGV utilization, $AAU(t)$

$$AU(t) = (AU_1(t), \ldots, AU_r(t), \ldots, AU_v(t)) \tag{17}$$

$$AU_r(t) = \frac{\sum_{i=1}^{n}\sum_{e=1}^{n_i}(UT_{i,e,r} + LT_{i,e,r}) \times YT_{i,e,r}(t)}{\sum_{r=1}^{v}\sum_{i=1}^{n}\sum_{e=1}^{n_i}(UT_{i,e,r} + LT_{i,e,r}) \times YT_{i,e,r}(t)} \tag{18}$$

$$AAU(t) = \frac{1}{v}\sum_{r=1}^{v} AU_r(t) \tag{19}$$

In Eq. 17, $AU_r(t)$ is the rth AGV utilization at decision point t, which can be calculated via Eq. 18.

Actions

In the FJS-AGV problem, the agent's actions at each decision point represents a scheduling behavior guiding the agent to select a suitable operation for processing. To ensure the completeness of the action space, nine scheduling behaviors are designed, as shown in Table 3.

The superiority of the DRL algorithm lies in its ability to make decisions based on real-time changes in workshop production states rather than relying on fixed rules or previous experiences. Through continuous learning and adjustment, DRL algorithms can automatically discover and optimize the best scheduling behaviors, thereby improving production efficiency and quality.

Rewards

After implementing an action, the environment provides a specific reward to the agent to appraise the quality of the action and incentivize the agent to improve. The goal of using the DRL algorithm to solve the FJS-AGV problem is to learn a policy to optimally schedule to minimize the makespan. To this end, we define the reward as the gap between the

partial solutions of two continuous decision points, as shown in Eq. 20.

$$r(t) = -(C_{\max}(t) - C_{\max}(t-1)) \tag{20}$$

When the discount factor $\gamma = 1$, the cumulative reward $R = -(\sum_{t=0}^{T} C_{\max}(T) - C_{\max}(0))$, where $C_{\max}(0) = 0$, so maximizing R and minimizing makespan is equivalent.

Exploration and exploitation

Exploration involves the agent trying out new actions in the hope of obtaining a higher reward, whereas exploitation involves selecting actions that are already known to generate the highest reward. Too much exploration may lead to inefficient decision-making, while too much exploitation can cause the agent to become stuck in suboptimal solutions. Exploration and exploitation are inherently contradictory, and finding the delicate balance between the two is crucial for maximizing the reward. The $\varepsilon - greedy$ algorithm is used in most DRL algorithms to improve the efficiency of exploration and exploitation. The action selection based on $\varepsilon - greedy$ is shown in Eq. 21.

$$a(t) = \begin{cases} \arg\max_{a \in A} Q(s_t, a_t), & with\ probability\ 1-\varepsilon; \\ random, & with\ probability\ \varepsilon; \end{cases} \tag{21}$$

Traditionally, ε is a constant, which can cause the agent to fall into a rigid pattern of actions too early, and the agent may become trapped and unable to adapt to environmental changes. To address this problem, we proposed an adaptive $\varepsilon - greedy$ strategy in which ε decreases with the number of learning steps. The ε updating rule is shown in Eq. 22.

$$\varepsilon(t) = \begin{cases} \varepsilon_{\max} - \frac{learn_step(t)}{learn_step_{\max}}, & \varepsilon \geq \varepsilon_{\min}; \\ \varepsilon_{\min}, & \varepsilon < \varepsilon_{\min}; \end{cases} \tag{22}$$

where $\varepsilon_{\max}$ and $\varepsilon_{\min}$ are the upper and bottom values of ε, respectively. $\varepsilon_{\min}$ can prevent $\varepsilon(t)$ from decreasing to zero, preventing the agent from exploiting the experiences only.

DQN topology and training

The HA-DQN algorithm employs a dual-network structure by using $OnlineNet$ to select actions, while a $TargetNet$ is used to evaluate the actions, which mitigates the problem of $Q - value$ overestimation during training. At initialization, $OnlineNet$ and $TargetNet$ share identical network structures and parameters, both comprising one input layer, three fully connected hidden layers, and one output layer. The number of nodes in the input and output layers corresponds to the number of state features and actions, respectively. Rectified linear unit (ReLU) activation functions are employed in the hidden layers. The online network Q_e is trained through the stochastic gradient method $RMSprop$ according to the error $E = \frac{1}{|A|}\sum_{i=1}^{|A|}(\hat{Q}_i - Q_i)^2$, while the target network $\hat{Q}$ is updated by $\hat{Q} = Q$ every twenty steps. In addition, an experience replay mechanism is incorporated, by storing and reusing past experiences, the agent can learn from a broader range of data, thus enhancing the model's generalizability.

Algorithm 1 Heuristic-assisted machine selection algorithm

Require: $O_{i,e}(t)$;
Ensure: M_k;
calculate the completion time $E_{i,e-1}$ of $O_{i,e-1}$;
record the current position $P_{J_i}(t)$ of J_i, at decision point t;
calculate the load transport start time $ST_{load} = \max\{E_{i,e-1}, RT_{V_s}(t) + UT_{P_{V_s}(t)\to P_{J_i}(t)}\}$ of operation $O_{i,e}(t)$;
find the compatible processing machine set $M_{i,e} \in M$ of $O_{i,e-1}$;
for M_l in $M_{i,e}$ **do**
  calculate the release time $RT_{M_l}(t)$ of M_l;
  calculate the load transport time $LT_{P_{J_i}(t)\to M_l}$ of J_i;
  calculate the processing start time $ST_{O_{i,e,l}}$ of operation $O_{i,e}(t)$ on machine M_l, $ST_{O_{i,e,l}} = \max\{RT_{M_l}(t), (ST_{load} + LT_{P_{J_i}(t)\to M_l})\}$;
endfor
generate a random number $\alpha \in [0, 1]$;
if $\alpha < 0.25$;
  randomly select a machine $M_k \in M_{i,e}$;
else
  The machine M_k with the smallest time of $ST_{O_{i,e,k}} + P_{i,e,k}$ is selected for processing $O_{i,e}(t)$;
endif

Heuristic-assisted machine and AGV selection algorithm

When the DQN algorithm is employed to solve the FJS-AGV problem, we can encounter situations where multiple actions are involved at each decision point t: 1) choose the optimal operation $O_{i,e}(t)$ for processing; 2) select the optimal machine M_k to process $O_{i,e}(t)$; and 3) select the optimal AGV V_s to transport $O_{i,e}(t)$; therefore, the agent's action is a triplet $(O_{i,e}(t), M_k, V_s)$. In environments characterized by multiple operations, machines, and AGVs, this leads to a rapid expansion of the action space, subsequently resulting in slower training speeds for the DQN. To mitigate this issue, we utilize the heuristic dispatch rule of Earliest Completion Time First (ECT) to assist DRL in selecting machines and AGVs, allowing the DQN to focus solely on operation selection, which can reduce the dimension of DRL action space heavily, thereby improving learning efficiency and convergence speed.

When using ECT to select a machine M_k for operation $O_{i,e}(t)$, the machine release time, transportation time, and machine process time are all considered, as shown in Eq. 23.

Algorithm 2 Heuristic-assisted AGV selection algorithm

Require: $O_{i,e}(t)$;
Ensure: V_s;
calculate the completion time $E_{i,e-1}$ of $O_{i,e-1}$;
record the current position $P_{J_i}(t)$ of J_i, at decision point t;
record the current positions $P_{AGVs}(t) = \{P_{V_1}(t), P_{V_2}(t), \ldots, P_{V_v}(t)\}$ of all AGVs, at decision point t;
calculate the release time $RT_{AGVs}(t) = \{RT_{V_1})(t), RT_{V_2}(t), \ldots, RT_{V_v}(t)\}$ of all AGVs, at decision point t;
for V_r in V **do**
    calculate the no-load transport time $UT_{P_{Vr}(t)\to P_{J_i}(t)}$ from $P_{Vr}(t)\ to\ P_{J_i}(t)$;
endfor
generate a random number $\alpha \in [0, 1]$;
if $\alpha < 0.25$;
    randomly select an AGV $V_s \in V$
else
    the AGV with the minimal time of $RT_{P_{V_s}(t)} + UT_{P_{V_s}(t)\to P_{J_i}(t)}$ is selected to transport $O_{i,e}$;
endif

For the specific implementation step, see ***Algorithm*** 1.

$$M_k = \underset{k\in 1,2,\ldots,m}{\arg\min}\ (RT_{M_k}(t) + LT_{P_{J_i}(t)\to M_k} + P_{i.e,k}) \tag{23}$$

where $RT_{M_k}(t)$ indicates the release time of machine M_k; $RT_{P_{J_i}(t)\to M_k}$ represents the transfer time of job J_i to machine M_k; $P_{i.e,k}$ is the process time of the eth operation of job J_i on machine M_k and $P_{J_i}(t)$ is the position of J_i at decision point t.

When using the ECT to select an AGV for operation $O_{i,e}(t)$, the AGV release time and the AGV unloading moving time are considered, as shown in Eq. 24; for the specific implementation step, see ***Algorithm*** 2.

$$V_s = \underset{s\in 1,2,\ldots,v}{\arg\min}\ (RT_{Vs}(t) + UT_{P_{Vs}(t)\to P_{J_i}(t)}) \tag{24}$$

where $RT_{Vs}(t)$ indicates the release time of V_s at decision point t; $UT_{P_{Vs}(t)\to P_{J_i}(t)}$ represents the unloading moving time of V_s to job J_i and $P_{Vs}(t)$ is the position of V_s at decision point t.

Solution representation and decoding

In the HA-DQN algorithm, the solution of the FJS-AGV problem can be expressed by an operation vector, a machine vector and an AGV vector, where each vector contains $\sum_{i=1}^{n} n_i$ elements. In the operation vector $O = \{\theta_1, \theta_2, \ldots, \theta_{n_i}\}$, element $\theta_i \in \{1, 2, \ldots, n\}$ is described by a job number. In the machine vector $\omega = \{\omega_1, \omega_2, \ldots, \omega_{n_i}\}$, element $\omega_i \in \{1, 2, \ldots, m\}$ is described by a machine number. In the AGV vector $\alpha = \{\alpha_1, \alpha_2, \ldots, \alpha_{n_i}\}$, element $\alpha_i \in \{1, 2, \ldots, v\}$ is described by an AGV number. In particular, ω_i and α_i are the machine and AGV allocated to operation θ_i.

The decoding procedure is essentially a process of creating a viable execution production scheme. Initially, according to the machine vector and operation vector, the assigned machine for each operation can be determined. The assigned AGV for each operation can be subsequently determined according to the AGV vector and operation vector. Ultimately, each operation is transported and processed by the assigned AGV and machine within the optimal available transport and processing time while adhering to the sequencing constraints defined by the operation vector. For example, the solution to the example problem fjsp3 discussed in Section 3.2 is presented in Fig. 4a, and an executable production scheme decoded from it is illustrated in Fig. 4b. Specifically, the processing procedure of J_3 is taken as an example (please see the green parts collected by the red arrows in Fig. 4b). Initially, all AGVs and jobs are located at the RW. V_2 first loads J_3 at the RW at time zero and then transports it to M_2 for the processing of $O_{3,1}$; after $O_{3,1}$ is completed, V_2 is then selected to load J_3 at M_2 and transport it to M_3 for the processing of $O_{3,2}$ and $O_{3,3}$. Finally, after $O_{3,3}$ is completed, V_2 is selected to load J_3 at M_3 and transport it to M_1 for the processing of $O_{3,4}$. The other job processing procedures are the same as those in J_3, and the complete Gantt scheme of all jobs is presented in Fig. 2.

Experiments and results

To test the performance and resilience of the HA-DQN algorithm, we conducted experiments on small-, medium- and large-scale benchmarks. All the experiments are conducted via Python 3.7 on a Windows 10 operating system running on an Intel Core i7 3.0 GHz PC with 10 GB of RAM.

Benchmarks and metrics

The small-scale benchmark, proposed by Deroussi and Norre [44], features 5 to 8 jobs with 13 to 20 operations processed on 8 machines. The medium- and large-scale benchmarks were developed by Seyed Mahdi Homayouni [45]; the former encompasses 5 to 12 jobs with 15 to 48 operations processed on 6 to 8 machines, whereas the latter involves 10 to 15 jobs with 100 to 225 operations processed on 11 to 18 machines. Each benchmark includes ten diverse instances, which are repeated 10 times to obtain the best results. Notably, all the instances utilize 2 AGVs for transportation.

The relative percentage deviation (RPD) proposed by Pan et al. [1] is used as the evaluation metric, as shown in Eq. 25.

$$RPD = (C_{\max}^{HA-DQN} - C_{\max}^{alg})/C_{\max}^{alg} \tag{25}$$

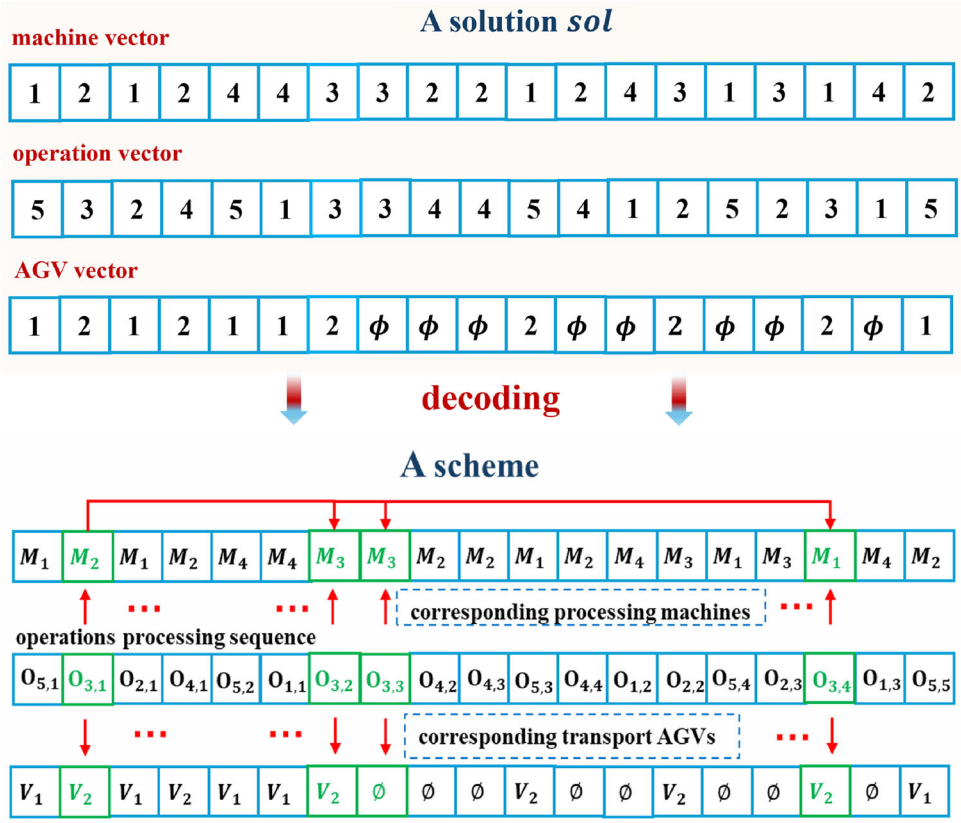


Fig. 4 Solution representation and decoding process, taking the problem fjsp3 as an example

where $C_{\max}^{HA-DQN}$ is the optimal $C_{\max}$ obtained by the proposed HA-DQN algorithm in this paper and $C_{\max}^{alg}$ is the optimal $C_{\max}$ obtained by algorithm alg.

Parameter calibration

The proposed HA-DQN algorithm involves four key parameters: ε in the $\varepsilon - decrease$ policy, the reward decay γ, the learning rate l, and the target network update threshold λ. To determine the appropriate setting of these parameters, a Taguchi orthogonal array approach [46] is employed. The levels of the four key parameters are as follows:

1. $\varepsilon = \{0.75, 0.85, 0.9, 0.95, 0.98\}$
2. $\gamma = \{0.5, 0.75, 0.9, 0.95\}$
3. $l = \{0.001, 0.005, 0.01\}$
4. $\lambda = \{5, 10, 20\}$

The four key parameters have $5*4*3*3 = 180$ combinations. We performed calibration experiments on the instance MFJST07 within the medium benchmark, involving 8 jobs, 28 operations, 8 machines and 2 AGVs. For each combination, the result is the average value of $C_{\max}$ obtained from 10 repeated experiments. The trends of the factor levels for the four parameters mentioned above are visualized in Fig. 5. Based on the analysis presented in Fig. 5, we select certain parameter values: $\varepsilon = 0.90$, $\gamma = 0.75$, $l = 0.005$, and $\lambda = 20$. The remaining parameters are set as follows: episode number $L = 2000$, memory size $MEM = 300$, and batch size $BS = 64$.

To further demonstrate the rationality of the parameter settings, taking the learning rate l as an example, we compared the convergence of the DQN algorithm under different learning rates. Figure 6 displays the convergence curves of the DQN algorithm when $\{l = 0.001, l = 0.005, l = 0.01\}$, while the other parameters are set to their optimal values and kept fixed. In this graph, the x-axis represents the number of learning steps, whereas the y-axis represents the value of the loss function. When $l = 0.005$, the convergence curve becomes smoother, and the value of the loss function gradually approaches zero.

Performance test on a small-scale benchmark

The small-scale benchmark was originally proposed by Deroussi and Norre [44] and contains 10 diversified instances designated fjsp1:10. The performance of HA-DQN is compared with that of the following methods: (1) an iterative local search algorithm, ILS [44]; (2) a shifting bottleneck algorithm, SBA [47]; (3) a tabu search algorithm, TS [48]; (4) a multistarting biased random key genetic algorithm, BRKGA [45]; and (5) a genetic algorithm, GA [49]. The comparative

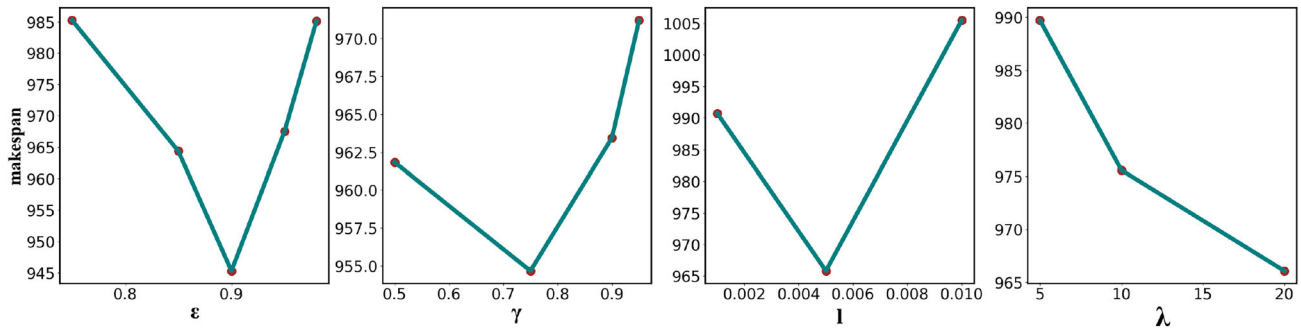


Fig. 5 Factor-level trend for parameters in the HA-DQN algorithm

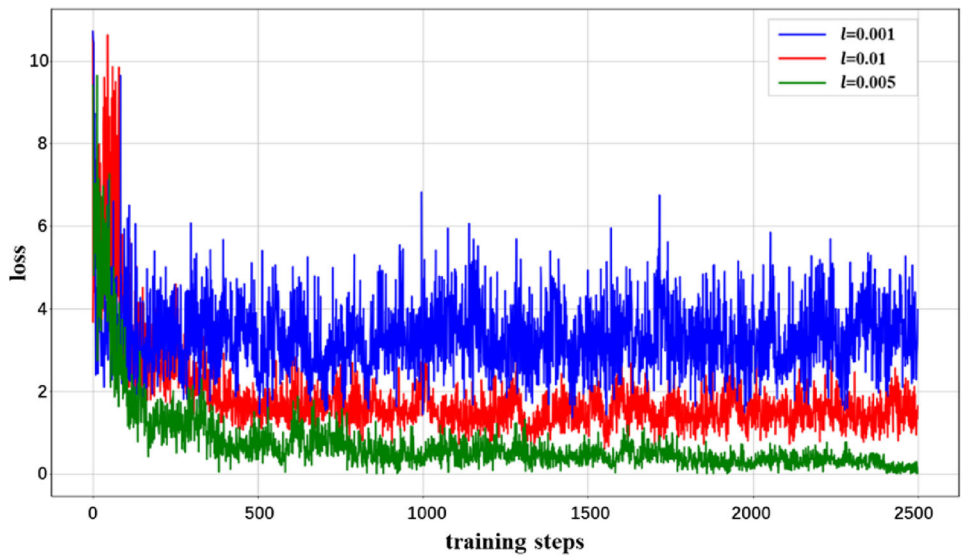


Fig. 6 Convergence curves of the HA-DQN algorithm under different parameter combinations

results for makespan and RPD for the abovementioned algorithms and the HA-DQN algorithm are given in Table 4, and the corresponding statistical charts are shown in Fig. 7.

As shown in Table 4, the HA-DQN algorithm can achieve optimal solutions in six instances and near-optimal solutions in the remaining four instances. The HA-DQN algorithm produces better solutions than the BRKGA algorithm does on 2 instances, fjsp4 and fjsp5, and performs better than the GA, SBA, TS, and ILS algorithms do on all instances. In the statistical chart, the orange and blue lines are used for comparing the solution qualities between the HA-DQN and ILS algorithms. It is evident that the quality of the solutions obtained by the HA-DQN and BRKGA are roughly identical, far surpassing the quality of the solutions obtained by other algorithms. In particular, the difference between HA-DQN and ILS is particularly notable, with the largest gap reaching 25.33%. This means that the solution obtained by the HA-DQN algorithm can reduce the production time by 25.33% compared with that obtained by the ILS algorithm. The superiority of the HA-DQN algorithm lies in its use of deep neural networks for learning and discovering production scheduling rules and patterns to optimize solutions. During training, the algorithm can extract features from large datasets and automatically adjust neural network weights. This enables it to better capture key information in the production process and optimize solutions accordingly. As a result, the HA-DQN algorithm offers significant advantages in addressing the FJS-AGV problem.

The Gantt chart in Fig. 8 illustrates an optimal scheduling solution for the example problem fjsp3, which includes 6 jobs with operation numbers of {2, 2, 2, 2, 4, 4}, 8 machines and 2 AGVs. The critical path, indicated by the red dashed boxes and blue arrows, is $\{O_{6,1}, O_{6,2}, O_{6,3}, O_{6,4}\}$, with the corresponding transport AGVs $\{V_2, V_2, V_2, V_2\}$. This critical path represents the longest path in the scheduling, and any delay in any of these activities will result in a delay in the overall project. Therefore, it is crucial to carefully manage and monitor the activities along this critical path to ensure that the project stays on schedule.

Table 4 RPD values between the HA-DQN algorithm and the comparative algorithms on the small-scale benchmark

Instance	M-J-O-A	LB	BRKGA		GA		SBA		TS		ILS		HA-DQN (ours)
			C_{max}	RPD	C_{max}	RPD	C_{max}	RPD	C_{max}	RPD	C_{max}	RPD	C_{max}
fjsp1	8-7-19-2	134	138	0.0000	144	−0.0417	156	−0.1154	160	−0.1375	160	−0.1375	138
fjsp2	8-6-15-2	114	118	0.0000	118	0.0000	124	−0.0484	128	−0.0781	138	−0.1450	118
fjsp3	8-6-16-2	120	120	0.0000	124	−0.0322	140	−0.1428	162	−0.2593	142	−0.1549	120
fjsp4	8-5-19-2	114	120	−0.0166	124	−0.049	132	−0.1061	126	−0.0635	138	−0.1450	118
fjsp5	8-5-13-2	94	96	−0.0208	94	0.0000	96	−0.0209	100	−0.0600	112	−0.1607	94
fjsp6	8-6-18-2	138	138	0.0000	144	−0.0417	148	−0.0676	152	−0.0921	158	−0.1266	138
fjsp7	8-8-19-2	110	112	0.0000	120	−0.0667	132	−0.1516	132	−0.1515	150	−0.2533	112
fjsp8	8-6-20-2	178	178	0.0000	179	−0.0056	191	−0.0681	188	−0.0532	197	−0.0965	178
fjsp9	8-5-17-2	144	144	0.0000	146	−0.0137	154	−0.0650	162	−0.1111	166	−0.1325	144
fjsp10	8-6-21-2	174	174	0.0000	182	−0.0440	192	−0.0938	186	−0.0645	188	−0.0745	174

Notes: the negative sign in RPD provide an indication of how much better the HA-DRL algorithm is compared to others; otherwise, it is not

Performance test on medium- and large-scale benchmarks

In this section, two algorithms, LAHC [10] and GA [49], which were recently adopted to solve the FJS-AGV problem in 2021 and 2022, respectively, are selected for comparison.

The LAHC algorithm, designed to solve the FJS-AGV problem by Seyed Mahdi Homayouni [45], is a local search-based algorithm that incorporates delayed comparisons and a scheduling history list to avoid local optimal solutions. When the medium benchmark instances are solved, the experimental results are shown in Table 5. The HA-DQN algorithm obtains the optimal solutions in five instances (MFJST01 to MFJST05) and the near-optimal solutions for the other five instances. HA-DQN obtains better solutions than the LAHC algorithm does in two instances (MFJST09 and MFJST10), but this superiority is not notable; the massive gap between HA-DQN and LAHC is only 1.22%. A Gantt chart of the instance MFJST07 obtained by the HA-DQN algorithm is presented in Fig. 9a. However, as the problem size increases, the optimization competence of the LAHC algorithm gradually deteriorates, whereas the superiority of the HA-DQN algorithm begins to emerge. As shown in Table 6, when solving large-scale problems, the HA-DQN algorithm always obtains a better solution than the LAHC algorithm does. Significantly, when dealing with the mt10xyz, mt10xxxt, and seti5cc instances, the HA-DQN algorithm can reduce C_{max} by 12.63%, 11.29%, and 10.97%, respectively. In addition, the LAHC algorithm is time-consuming, and when solving large-scale FJS-AGV problems, it takes hours to search for the optimal solution. For example, the LAHC algorithm requires 2.5 h to find a feasible solution when solving the seti5xyz instance.

The GA algorithm was recently modified by Imran Ali Chaudhry [49] for solving the FJS-AGV problem, in which adapted improvements were designed for encoding, decoding, crossover and mutation operators. The experimental results in Table V show that the GA only obtains the optimal solution for instances with fewer operations, such as MFJST01 to MFJST03. When the number of operations increases, its optimizing ability is limited and inferior to that of the HA-DQN algorithm. Specifically, for example, for MFJST08, HA-DQN saves 23.39% of C_{max} compared with the GA algorithm. Figure 9b depicts the makespan curves obtained by the HA-DQN, GA and LAHC algorithms when different instances within the medium benchmark are solved.

Figure 10 illustrates the solving process of the HA-DQN algorithm on instance seti5xyz within the large benchmark. In Fig. 10a, the makespan values obtained by the HA-DQN agent at each learning step are depicted, indicating that the DQN agent maintains active and effective learning during its interaction with the environment until it reaches a near-optimal solution. Figure 10b presents the correspond-

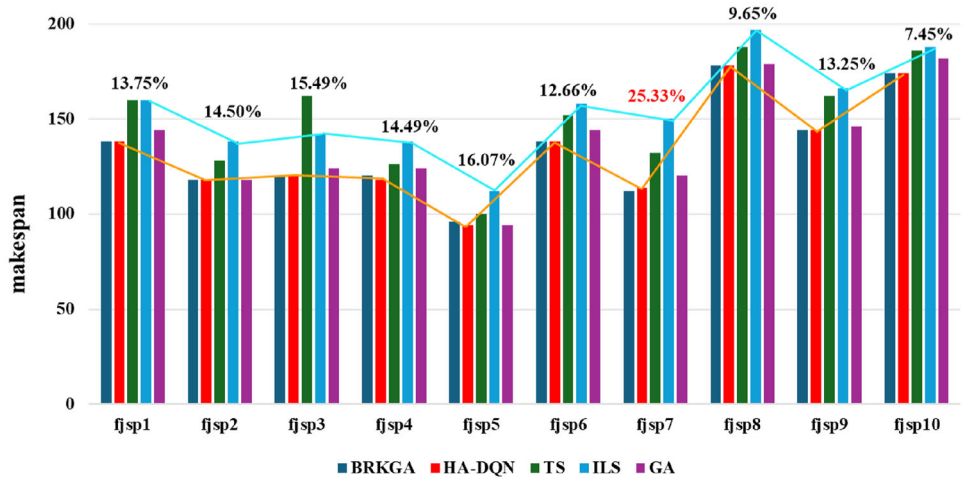
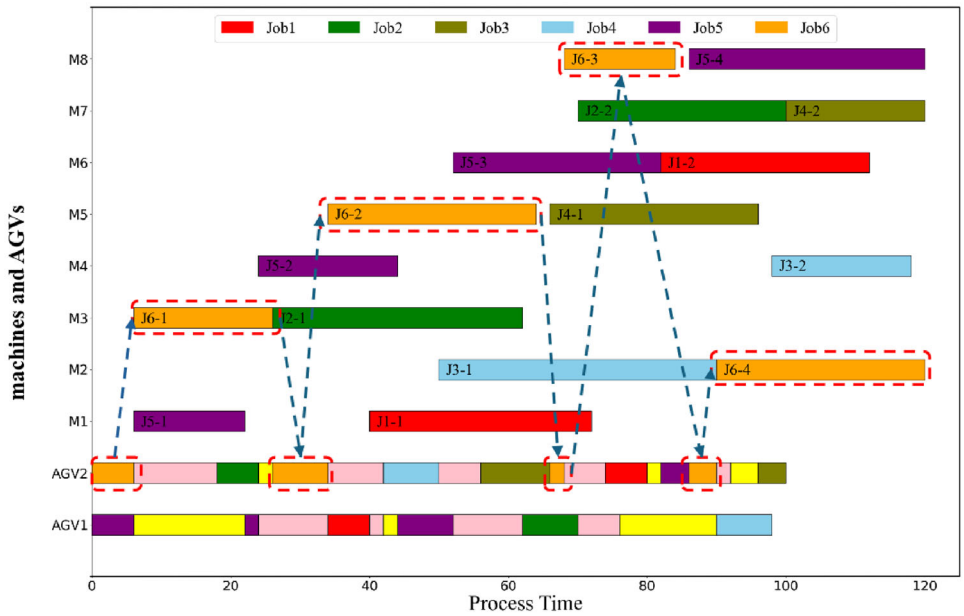


Fig. 7 C_{max} obtained via the HA-DQN, BRKGA, GA, SBA, TS, and ILS algorithms



Notes: for AGVs, pink rectangles represent the no-load action; yellow rectangles represent the idle and standby state; all others represent the load action.

Fig. 8 Gantt chart of the example problem fjsp3 obtained by HA-DQN

Table 5 RPD values between the HA-DQN algorithm and the comparative algorithms when dealing the medium benchmark

Instance	M-J-O-A	LB	LAHC		GA		HA-DQN (ours)
			C_{max}	RPD	C_{max}	RPD	C_{max}
MFJST01	6-5-15−2.2	485	485	0.0000	485	0.0000	485
MFJST02	7-5-15−2.6	463	468	−0.0107	463	0.0000	463
MFJST03	7-6-18−2.7	482	482	0.0000	482	0.0000	482
MFJST04	7-7-21−2.7	576	576	0.0000	584	−0.0137	576
MFJST05	7-7-21−2.6	532	532	0.0000	542	0.0184	532
MFJST06	7-8-24−2.6	652	652	0.0030	640	0.0218	654
MFJST07	7-8-32−2.4	780	907	0.0187	1016	−0.0905	924
MFJST08	8-9-36−2.4	NA	918	0.0131	1214	−0.2339	930
MFJST09	8-11-44−2.3	NA	1181	−0.0025	1415	−0.1674	1178
MFJST10	8-12-48−2.3	NA	1310	−0.0122	1613	−0.1977	1294

Notes: the negative sign in RPD provide an indication of how much better the HA-DRL algorithm is compared to others; otherwise, it is not

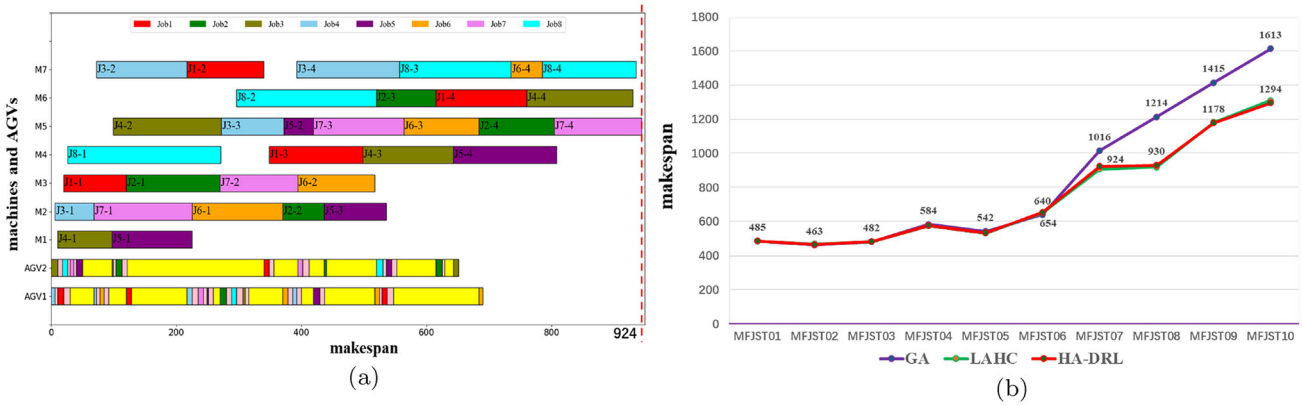


Fig. 9 Scheduling results on the medium benchmark. (**a**) Gantt chart of MFJST07 obtained by HA-DQN. (**b**) Solving capacity comparison between the HA-DQN, LAHC and GA algorithms on different instances

Table 6 RPD values between the HA-DQN algorithm and the comparative algorithms when dealing the large benchmark

Instance	M-J-O-A	LB	LAHC(H=1000)			HA-DQN (ours)	
			C_{max}	T(seconds)	RPD	C_{max}	T(seconds)
mt10ct1	11-10-100−1.1	715	992	382.7	−0.0232	969	81.36
mt10cc	12-10-100−1.2	729	973	552.7	−0.0411	933	82.94
mt10x	11-10-100−1.1	719	991	457.4	−0.0495	942	81.22
mt10xxxt	13-10-100−1.3	733	983	546.1	−0.1129	872	82.21
mt10xyz	13-10-100−1.3	703	934	651.1	−0.1263	816	81.88
setb4c9	11-15-150−1.1	848	1005	1275.7	−0.0467	958	123.64
setb4cc	12-15-150−1.2	848	979	1704.2	−0.0337	946	123.95
setb4xyz	13-15-150−1.3	859	979	2000.9	−0.0480	932	123.48
seti5c12	16-15-225−1.1	1045	1361	8234.9	−0.1058	1217	185.83
seti5cc	17-15-225−1.1	1051	1358	9516.7	−0.1097	1209	185.62
seti5xyz	18-15-225−1.2	1035	1347	8905.7	−0.1031	1208	186.75

Notes: the negative sign in RPD provide an indication of how much better the HA-DRL algorithm is compared to others; otherwise, it is not

ing Gantt chart of this near-optimal scheduling solution, revealing that the limited number of AGVs causes congestion and transportation delays, resulting in idle time for many machines, thereby posing challenges in achieving a smaller makespan.

After a thorough analysis of the three algorithms' characteristics and a careful examination of the experimental results, three conclusions can be drawn: (1) The LAHC and GA algorithms rely heavily on numerous iterations to achieve the optimal solution and are highly dependent on the parameters. Consequently, when tackling large-scale FJS-AGV, the solution space is tremendous, their optimizing capability is limited, and the optimal solution cannot be found. (2) The HA-DQN algorithm can learn the optimal solution adaptively by interacting with the environment, and its performance can be improved through continuous practice and experimentation during interactions. Therefore, the HA-DQN algorithm achieves better solutions in dealing with complex and high-dimensional problems, and this characteristic helps it solve large-scale FJS-AGV problems effectively.

Generalization testing of the HA-DQN algorithm

To ascertain the generalizability of the HA-DQN algorithm, we migrated the HA-DQN model trained on large-scale benchmark in section 5.4 to an unseen FJS-AGV benchmark dataset. This dataset encompasses ten distinctively configured instances, namely MK01 to MK10, which were initially proposed by Brandimarte [50] and subsequently improved by Homayouni and Fontes [15] through incorporating transportation time. These datasets encompass scenarios with 10, 15, and 20 jobs, 55 to 240 operations, and 4 to 15 machines. Each instance was subjected to ten iterative trials, with the average performance recorded. A comparative evaluation of these findings was undertaken against the performance of the LAHC algorithm as outlined in [15]. A graphical representation of these comparative results is presented in Fig. 11.

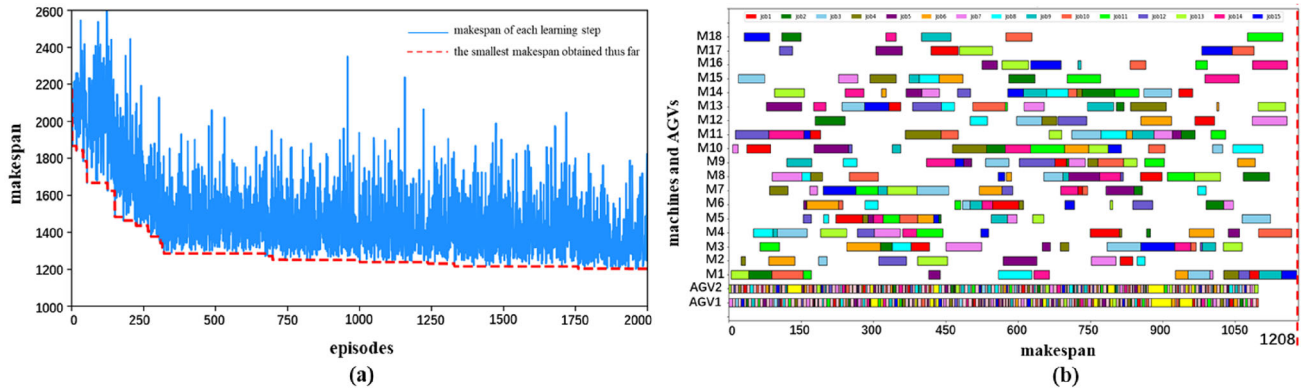
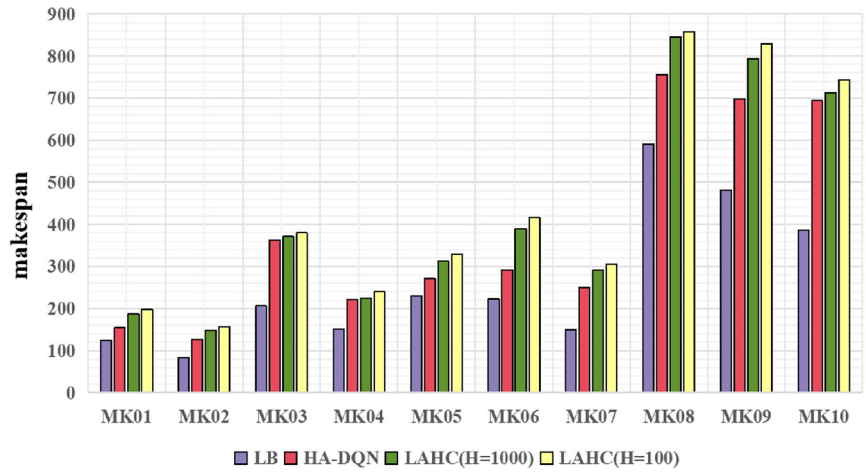


Fig. 10 Scheduling results of the instance seti5xyz by HA-DQN. (**a**) Makespan values at each learning step. (**b**) Gantt chart of the optimal schedule solution



Instance \ CPU time (s)	HA-DQN	LAHC (H=100)	LAHC (H=1000)
MK01	1.03	9.10	209.80
MK02	16.06	11.08	241.50
MK03	39.23	291.10	2678.90
MK04	4.26	24.70	371.80
MK05	4.96	28.10	483.70
MK06	6.87	336.50	3544.00
MK07	5.72	49.50	582.70
MK08	52.79	621.80	6809.20
MK09	92.37	838.50	8969.90
MK10	95.26	854.80	10768.30

Note: CPU time means the time to obtain optimal scheduling solutions by the algorithms

(b)

Fig. 11 (**a**) $C_{\max}$ obtained by the trained HA-DQN model and LAHC (H=1000) and LAHC (H=100) algorithms. (**b**) CPU time required by the HA-DQN model and LAHC (H=1000) and LAHC (H=100) algorithms to solve each instance, respectively

Based on the experimental results, we analyze the generalizability of the HA-DQN model from three aspects. First, the experimental structure demonstrates that the HA-DQN algorithm achieves excellent scheduling solutions across instances MK01 to MK10, indicating that the trained HA-DQN model can smoothly transfer to solve production scheduling problems in unknown scenarios. Second, the quality of the solution obtained by the HA-DQN model is superior to that of the LAHC algorithm, particularly in the MK07 scenario, where the completion time of the scheduling solution obtained by the HA-DQN algorithm is reduced by 14.15% compared with that of the LAHC algorithm. Finally, the HA-DQN algorithm has higher time efficiency. For example, the trained HA-DQN model only requires 95 s to solve the most complex case, MK10, which is significantly less than the 10768.30 s needed by the LAHC. The experimental results indicate that the HA-DQN algorithm demonstrates good generalizability in both problem-solving ability and time efficiency for unseen FJS-AGV problems.

Ablation experiments

To verify the positive impact of heuristic dispatch rule on the performance of the DQN algorithm when solving the FJS-AGV problem, this section conducts a set of ablation experiments based on the benchmark instances (MK01-MK10) used in Section 5.5. The experiments encompass three schemes: *scheme 1* utilizes the HA-DQN algorithm to address the FJS-AGV problem, *scheme 2* and *3* apply the traditional DQN and PPO algorithm to the same problem, respectively. In *schemes 1*, the HA-DQN uses heuristic dispatch rules to do machine and AGV selection, while the DQN focus solely on operation selection. In *schemes 2 and 3*, the DQN and PPO need to selects operation, machine and AGV

Table 7 C_{max} obtained by the HA-DQN, DQN and PPO algorithms

Instance	M-J-O-A	LB	HA-DQN (ours)			DQN				PPO			
			Best	Mean	Cov	Best	Mean	Cov	RPD	Best	Mean	Cov	RPD
MK01	6-10-55−2.1	125.00	153	154.50	0.0052	153	155.70	0.0114	−0.0077	153	155.90	0.0105	−0.0089
MK02	6-10-58−4.1	83.00	122	125.80	0.0103	122	128.60	0.0205	−0.0217	122	128.00	0.0187	−0.0171
MK03	8-15-150−3.0	206.00	356	359.50	0.0067	358	364.30	0.0121	−0.01314	357	364.70	0.0119	−0.0142
MK04	8-15-90−1.9	152.00	213	220.80	0.0145	215	222.60	0.0293	−0.0080	216	223.10	0.0202	−0.0103
MK05	4-15-106−1.2	231.00	251	267.80	0.0201	276	287.36	0.0226	−0.0680	271	281.27	0.0241	−0.0478
MK06	15-10-150−3.3	222.80	329	336.72	0.0156	354	365.00	0.0162	−0.0774	335	345.45	0.0189	−0.0252
MK07	5-20-100−2.8	150.00	243	249.80	0.0172	243	254.80	0.0206	−0.0196	244	252.05	0.0190	−0.0089
MK08	10-20-225−1.4	591.50	723	755.60	0.0197	752	775.60	0.0286	−0.0257	752	773.15	0.0208	−0.0226
MK09	10-20-240−2.5	481.80	681	691.00	0.0083	695	708.80	0.0118	−0.0251	693	703.41	0.0109	−0.0176
MK10	10-20-240−3.0	386.80	657	669.50	0.0102	662	692.90	0.0319	−0.0337	664	691.30	0.0234	−0.0315

Notes: (1) the negative sign in RPD provide an indication of how much better the HA-DRL algorithm is compared to others; otherwise, it is not. (2) Cov-coefficient of variation

simultaneously. Each scheme is subjected to ten experimental trials, with the results recorded in Table 7.

As illustrated in Table 7, although the HA-DQN, DQN, and PPO algorithms are all capable of obtaining near-optimal solutions when addressing the FJS-AGV problem, there are differences in the quality of the solutions. First, the solutions obtained by the HA-DQN algorithm are consistently superior to (or equivalent to) those obtained by the DQN and PPO algorithms. For example, the average C_{max} value achieved by HA-DQN is reduced by 7.74% compared to that of the DQN on MK06 and 4.78% compared to that of the PPO on MK02. Then, the HA-DQN algorithm exhibits better stability when solving the FJS-AGV problem, as evidenced by the coefficients of variation (Cov) in their solutions. For instance, when solving instance MK10, the Cov value of HA-DQN's ten experimental results is only 0.0102, whereas the Cov values of DQN and PPO are 0.0319 and 0.0234, respectively. This comparative analysis demonstrates that the time-based heuristic scheduling rules can assist the DQN algorithm in making more precise matches of operations, machines, and AGVs at each decision point, reducing the randomness associated with the DRL algorithm's learning through random combinations of the operations, machines and AGVs when interacting with the environment, thereby enhancing the efficiency and stability of the HA-DQN algorithm in solving the FJS-AGV problems. Figure 12 presents the box plots of the experimental results from ten trials for six sets of data: MK01, MK03, MK04, MK06, MK09, and MK10, which provide a more intuitive representation of the solution quality and distribution range obtained by the HA-DQN, DQN, and PPO algorithms when solving the FJS-AGV problem.

Compare with DRL methods

This section further substantiates the generalizability and efficiency of the HA-DQN method by benchmarking it against the most representative DRL algorithms tailored for the FJS-AGV problems. Comparsion algorithms and corresponding data are sourced from work [51], where the dataset closely resembles the one presented in Section 5.6, with the exception that the number of AGVs for each instance follows a uniform distribution $U(0.8m, 1.2m)$. The baseline algorithms include:

- PPO+GNNs: a DRL-based algorithm characterized by employing GNNs to automatically extract environmental state features. Originally designed for FJSP only [52], it was later enhanced with the Nearest Vehicle Selection method for AGV selection by work [51], extending its applicability to the FJS-AGV problem.
- HGS (Heterogeneous Graph Scheduler): the pioneering DRL method capable of obtaining near-optimal solutions irrespective of the scale of operations, machines, and AGVs, thereby effectively addressing the challenge of scale generalization in FJS-AGV problem.

Table 8 compares the scheduling solutions obtaiend by HA-DQN algorithm with that of PPO+GNNs and HGS. Obviously, our approach consistently outperforms the comparison algorithms across all instances. When comparing with the PPO+GNNs algorithm, the RPD reaches 42.28%. Furthermore, when comparing with the HGS algorithm, the HA-DQN method achieves a 38.79% reduction in C_{max} in the case of MKT07. The experimental results demonstrate that the HA-DQN algorithm has better problem-solving capabilities and generalizability compared to PPO+GNNs and HGS algorithm.

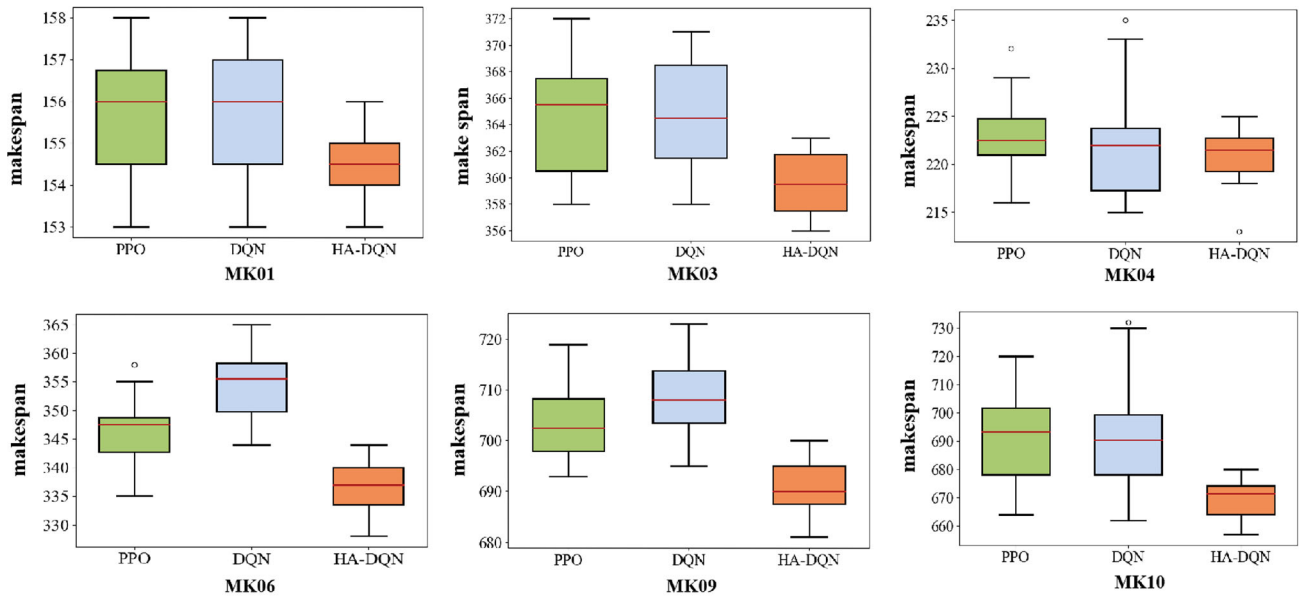


Fig. 12 Box plots of the makespan obtained by the three algorithms in different instances

Table 8 C_{max} obtained by the HA-DQN, PPO+GNNs and HGS algorithms

Instance	J-M-A	LB	PPO+GNNs			HGS			HA-DQN (ours)	
		(No transportation)	C_{max}	T(s)	RPD	C_{max}	T(s)	RPD	C_{max}	T(s)
MKT01	10-6-6	36	131	0.34	−0.2595	153	0.64	−0.3660	97	0.64
MKT02	10-6-6	24	123	0.34	−0.4228	104	0.64	−0.3173	71	0.62
MKT03	15-8-8	204	306	0.96	−0.2320	267	1.67	−0.1199	235	1.27
MKT04	15-8-8	48	160	0.52	−0.1938	139	1.00	−0.0719	129	0.96
MKT05	15-4-4	168	388	0.62	−0.3325	374	1.18	−0.3075	259	1.05
MKT06	10-15-15	33	213	0.94	−0.2582	217	1.69	−0.2719	158	1.7
MKT07	20-5-5	133	406	0.59	−0.4754	348	1.12	−0.3879	213	1.07
MKT08	20-10-10	523	819	1.42	−0.1832	812	2.56	−0.1761	669	2.14
MKT09	20-10-10	299	682	1.52	−0.2639	529	2.72	−0.0510	502	2.44
MKT10	20-15-15	165	531	1.52	−0.3277	409	2.72	−0.1271	357	2.66

Notes: the negative sign in RPD provide an indication of how much better the HA-DRL algorithm is compared to others; otherwise, it is not

Conclusion

This work studies the flexible job shop machine and AGV cooperative scheduling problem, which is briefly referred to as the FJS-AGV problem. The objective is to minimize the makespan. The FJS-AGV problem is an NP-hard problem involving 1) allocating transportation tasks to AGVs; 2) sequencing transportation tasks on each AGV; 3) assigning processing tasks to machines; and 4) sequencing processing tasks on each machine. The solution space and computational complexity are much greater than those of the classical FJS scheduling problem due to the additional scheduling dimensions introduced by the AGVs. To address the FJS-AGV problem, a novel heuristic-assisted deep reinforcement learning algorithm, called HA-DQN, is proposed. Initially, the FJS-AGV problem is formulated as a Markov decision process. Next, the DQN algorithm is employed to determine the optimal operation to be executed at each decision point. The heuristic dispatch rule of Earliest Completion Time First is subsequently used to select a suitable machine and AGV for executing and transporting that operation while considering the current production environment information, such as machine release time, AGV release time and moving distances. Finally, an adaptive $\varepsilon - greedy$ strategy is proposed to balance the exploration-exploitation operators, further helping the HA-DQN algorithm to converge faster to optimal solutions while maintaining the diversity of the action space.

Through detailed experiments on instances in small-, medium- and large-scale benchmarks, the feasibility and effectiveness of the HA-DQN algorithm in solving the FJS-

AGV problem is demonstrated. During the problem-solving process, the HA-DQN algorithm outperforms traditional heuristic algorithms in terms of the solution speed and solution quality. In particular, when solving large-scale FJS-AGV problems, the HA-DQN algorithm reduces the makespan by 12.63% compared with the best-performing LAHC algorithm, with a significantly shorter solution time. The experimental results indicate that the HA-DQN algorithm is a promising approach for solving large-scale FJS-AGV problems, and it is worth devoting more effort and time to exploring its potential in addressing other complex scheduling problems in flexible production systems. In addition, the generalization experiments further prove that the HA-DQN algorithm has good migration ability, the well trained policy network can efficiently solve the unseen scheduling instances. Currently, our research considers only static scheduling when solving the FJS-AGV problem, which limits the application of the proposed HA-DQN algorithm in dynamic scheduling environments. In the future, we plan to expand our work on the FJS-AGV problem to incorporate complex constraints such as dynamic scheduling, multiobjective optimization, and energy-aware scheduling. Furthermore, we intend to explore the integration of DRL with effective evolutionary algorithms. Additionally, we will investigate learning strategies and neural network architectures to improve the performance of the proposed algorithm.

Author Contributions All the authors contributed to the study conception and design. Material preparation, data collection and analysis were performed by [Xiaoting Dong] and [Guangxi Wan]. The first draft of the manuscript was written by [Xiaoting Dong], and all the authors commented on previous versions of the manuscript. All the authors read and approved the final manuscript.

Funding This work was supported by the National Natural Science Foundation of China [U24A20277, 2467301, 2367301, 2267205], Natural Science Foundation of Liaoning Province [2024-MSBA-83], the National Program for Funded Postdoctoral Researchers [GZB20230805], the State Key Laboratory of Robotics of China [2023-Z15], and Fundamental Research Project of SIA, [2024JC1K10]

Data Availability Data available on request from the authors.

Declarations

Financial interest or non-financial interest All authors certify that they have no affiliations with or involvement in any organization or entity with any financial interest or non-financial interest in the subject matter or materials discussed in this manuscript.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Pan Z, Wang L, Zheng J, Chen J-F, Wang X (2023) A learning-based multipopulation evolutionary optimization for flexible job shop scheduling problem with finite transportation resources. IEEE Trans Evol Comput 27(6):1590–1603. https://doi.org/10.1109/TEVC.2022.3219238
2. Xu WWWYeaW (2023) Flexible job-shop scheduling method based on interval grey processing time. Appl Intell 53:14876–14891. https://doi.org/10.1007/s10489-022-04213-9
3. Wan G, Dong X, Dong Q, He Y, Zeng P (2022) Context-aware scheduling and control architecture for cyber-physical production systems. J Manuf Syst 62:550–560. https://doi.org/10.1109/TEVC.2022.3219238
4. Arulkumaran K, Deisenroth MP, Brundage M, Bharath AA (2017) Deep reinforcement learning: A brief survey. IEEE Signal Process Mag 34(6):26–38. https://doi.org/10.1109/MSP.2017.2743240
5. Zhao L, Fan J, Zhang C, Shen W, Zhuang J (2023) A drl-based reactive scheduling policy for flexible job shops with random job arrivals. IEEE Trans Autom Sci Eng 8:1–12. https://doi.org/10.1109/TASE.2023.3271666
6. Gu CMY, Wang L (2023) A self-learning discrete salp swarm algorithm based on deep reinforcement learning for dynamic job shop scheduling problem. Appl Intell 53:18925–18958. https://doi.org/10.1007/s10489-023-04479-7
7. Caumond A, Lacomme P, Moukrim A, Tchernev N (2009) An milp for scheduling problems in an fms with one vehicle. Eur J Oper Res 199(3):706–722. https://doi.org/10.1016/j.ejor.2008.03.051
8. Novas JM, Henning GP (2014) Integrated scheduling of resource-constrained flexible manufacturing systems using constraint programming. Expert Syst Appl 41(5):2286–2299. https://doi.org/10.1016/j.eswa.2013.09.026
9. Ham A (2020) Transfer-robot task scheduling in flexible job shop. J Intell Manuf 31(7):1783–1793. https://doi.org/10.1007/s10845-020-01537-6
10. Homayouni SM, Fontes DBMM (2021) Production and transport scheduling in flexible job shop manufacturing systems. J Global Optim 79(2):463–502. https://doi.org/10.1007/s10898-021-00992-6
11. Kumar MVS, Janardhana R, Rao CS (2011) Simultaneous scheduling of machines and vehicles in an fms environment with alternative routing. Int J Adv Manuf Technol 53:339–351. https://doi.org/10.1007/s00170-010-2820-2
12. Nouri HE, Belkahla Driss O, Ghédira K (2016) Simultaneous scheduling of machines and transport robots in flexible job shop environment using hybrid metaheuristics based on clustered holonic multiagent model. Comput Ind Eng 102:488–501. https://doi.org/10.1016/j.cie.2016.02.024
13. Espinouse M-L, Pawlak G, Sterna M (2017) Complexity of scheduling problem in single-machine flexible manufacturing system with cyclic transportation and unlimited buffers. J Optim Theory Appl 173(3):1042–1054. https://doi.org/10.1007/s10957-016-1056-1
14. Dang Q, Nguyen CT, Rudová H (2019) Scheduling of mobile robots for transportation and manufacturing tasks. J Heurist 25:175–213. https://doi.org/10.1007/s10732-018-9391-z
15. Tan W, Yuan X, Huang G, Liu Z (2021) Low-carbon joint scheduling in flexible open-shop environment with constrained

automatic guided vehicle by multi-objective particle swarm optimization. Appl Soft Comput 111:107695. https://doi.org/10.1016/j.asoc.2021.107695
16. Cheng L, Tang Q, Zhang Z (2020) Data mining for fast and accurate makespan estimation in machining workshops. Robot Comput-Integr Manuf 32:483–500. https://doi.org/10.1007/s10845-020-01585-y
17. Yan J, Liu Z, Zhang C, Zhang T, Zhang Y, Yang C (2021) Research on flexible job shop scheduling under finite transportation conditions for digital twin workshop. Robot Comput-Integr Manuf 72:102198. https://doi.org/10.1016/j.rcim.2021.102198
18. Chen K, Bi L, Wang W (2022) Research on integrated scheduling of agv and machine in flexible job shop. J Syst Simul 34(3):483–500. https://doi.org/10.16182/j.issn1004731x.joss.20-0796
19. Yuan M, Li Y, Pei F, Gu W (2021) Dual-resource integrated scheduling method of agv and machine in intelligent manufacturing job shop. J Central South Univ 28:2423–2435. https://doi.org/10.1007/s11771-021-4777-8
20. Wen X (2023) An effective hybrid algorithm for joint scheduling of machines and agvs in flexible job shop. Meas Control 56:1582–1598. https://doi.org/10.1177/00202940231173750
21. He L, Chiong R, Li W, Budhi GS, Zhang Y (2022) A multi-objective evolutionary algorithm for achieving energy efficiency in production environments integrated with multiple automated guided vehicles. Knowl-Based Syst 243:108315. https://doi.org/10.1016/j.knosys.2022.108315
22. Pan Z, Wang L, Wang J, Zhang Q (2024) A bi-learning evolutionary algorithm for transportation-constrained and distributed energy-efficient flexible scheduling. IEEE Trans Evol Comput 8:1–15. https://doi.org/10.1109/TEVC.2024.3354850
23. Zhang F, Li R, Gong W (2024) Deep reinforcement learning-based memetic algorithm for energy-aware flexible job shop scheduling with multi-agv. Comput Ind Eng 189:109917. https://doi.org/10.1016/j.cie.2024.109917
24. Silver D, Hubert T, Schrittwieser J, Antonoglou I, Lai M, Guez A (2017) Mastering chess and shogi by self-play with a general reinforcement learning algorithm. arXiv preprint https://doi.org/10.48550/arXiv.1712.01815
25. Vinyals O, Babuschkin I et al (2019) Grandmaster level in starcraft ii using multi-agent reinforcement learning. Nature 575(7782):350–354. https://doi.org/10.1038/s41586-019-1724-z
26. Lin C-C, Deng D-J, Chih Y-L, Chiu H-T (2019) Smart manufacturing scheduling with edge computing using multiclass deep q network. IEEE Trans Industr Inf 15(7):4276–4284. https://doi.org/10.1109/TII.2019.2908210
27. Palombarini A, J, Martínez EC (2018) Automatic generation of rescheduling knowledge in socio-technical manufacturing systems using deep reinforcement learning. In: 2018 IEEE Biennial Congress of Argentina (ARGENCON), pp. 1–8. https://doi.org/10.1109/ARGENCON.2018.8646172
28. Liu C-L, Chang C-C, Tseng C-J (2020) Actor-critic deep reinforcement learning for solving job shop scheduling problems. IEEE Access 8:71752–71762. https://doi.org/10.1109/ACCESS.2020.2987820
29. Han B-A, Yang J-J (2020) Research on adaptive job shop scheduling problems based on dueling double dqn. IEEE Access 8:186474–186495. https://doi.org/10.1109/ACCESS.2020.3029868
30. Wang L, Hu X (2021) Dynamic job-shop scheduling in smart manufacturing using deep reinforcement learning. Comput Netw 190:107969. https://doi.org/10.1016/j.comnet.2021.107969
31. Gebreyesus G, Fellek G, Farid AM, Fujimura S, Yoshie O (2023) Gated-attention model with reinforcement learning for solving dynamic job shop scheduling problem. IEEJ Trans Electr Electron Eng 18:8. https://doi.org/10.1002/tee.23788
32. Zhang M, Lu Y, Hu Y, Amaitik N, Xu Y (2022) Dynamic scheduling method for job-shop manufacturing systems by deep reinforcement learning with proximal policy optimization. Sustainability 14:5177. https://doi.org/10.3390/su14095177
33. Wu X, Yan X, Guan D, Wei M (2024) A deep reinforcement learning model for dynamic job-shop scheduling problem with uncertain processing time. Eng Appl Artif Intell 131:107790. https://doi.org/10.1016/j.engappai.2023.107790
34. Aihong S, Qi L (2024) Deep reinforcement learning for solving the joint scheduling problem of machines and agvs in job shop. Control Decis 39:1. https://doi.org/10.13195/j.kzyjc.2022.1821
35. Liu R, Piplani R, Toro C (2023) A deep multi-agent reinforcement learning approach to solve dynamic job shop scheduling problem. Comput Oper Res 159:106294. https://doi.org/10.1016/j.cor.2023.106294
36. Luo S (2020) Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. Appl Soft Comput 91:106208. https://doi.org/10.1016/j.asoc.2020.106208
37. Luo S, Zhang L, Fan Y (2021) Dynamic multi-objective scheduling for flexible job shop by deep reinforcement learning. Comput Ind Eng 159:107489. https://doi.org/10.1016/j.cie.2021.107489
38. Lei K, Guo P, Zhao W, Wang Y, Qian L, Meng X, Tang L (2022) A multi-action deep reinforcement learning framework for flexible job-shop scheduling problem. Expert Syst Appl 205:117796. https://doi.org/10.1016/j.eswa.2022.117796
39. Zhang J-D, He Z, Chan W-H, Chow C-Y (2023) Deepmag: Deep reinforcement learning with multi-agent graphs for flexible job shop scheduling. Knowl-Based Syst 259:110083. https://doi.org/10.1016/j.knosys.2022.110083
40. Li R, Gong W, Wang L, Lu C, Dong C (2024) Co-evolution with deep reinforcement learning for energy-aware distributed heterogeneous flexible job shop scheduling. IEEE Trans Syst Man Cybern Syst 54(1):201–211. https://doi.org/10.1109/TSMC.2023.3305541
41. Zhang L, Feng Y, Xiao Q, Xu Y, Li D, Yang D, Yang Z (2023) Deep reinforcement learning for dynamic flexible job shop scheduling problem considering variable processing times. J Manuf Syst 71:257–273. https://doi.org/10.1016/j.jmsy.2023.09.009
42. Zhao L, Fan J, Zhang C, Shen W, Zhuang J (2023) A drl-based reactive scheduling policy for flexible job shops with random job arrivals. IEEE Trans Autom Sci Eng 8:1–12. https://doi.org/10.1109/TASE.2023.3271666
43. Yuan E, Wang L, Cheng S, Song S, Fan W, Li Y (2024) Solving flexible job shop scheduling problems via deep reinforcement learning. Expert Syst Appl 245:123019. https://doi.org/10.1016/j.eswa.2023.123019
44. Deroussi L, Norre S (2010) Simultaneous scheduling of machines and vehicles for the flexible job shop problem. In: Proceedings of the International Conference on Metaheuristics and Nature Inspired Computing
45. Homayouni SM, Fontes DBMM (2021) Production and transport scheduling in flexible job shop manufacturing systems. J Glob Optim 79:463–502. https://doi.org/10.1007/s10898-021-00992-6
46. Nostrand RCV (2002) Design of experiments using the taguchi approach: 16 steps to product and process improvement. Technometrics 44:289. https://doi.org/10.1198/004017002320256440
47. Qiao Zhang M-AM (2013) Metaheuristics for job shop scheduling with transportation. In: Metaheuristics for Production Scheduling, pp. 465–493
48. Zheng Y, Xiao Y, Seo Y (2014) A tabu search algorithm for simultaneous machine/agv scheduling problem. Int J Prod Res 52:5748–5763. https://doi.org/10.1080/00207543.2014.910628
49. Chaudhry IA, Rafique AF, Elbadawi I, Aichouni M, Usman M, Boujelbene M, Boudjemline A (2022) Integrated scheduling of machines and automated guided vehicles (agvs) in flexible job shop environment using genetic algorithms. Int J Ind Eng Comput 13(3):343–362. https://doi.org/10.5267/j.ijiec.2022.2.002

50. Brandimarte: Routing and scheduling in a flexible job shop by tabu search. Ann Oper Res 41:157–183 (1993) https://doi.org/10.1007/BF02023073
51. Moon S (2024) Learning-enabled flexible job-shop scheduling for scalable smart manufacturing. J Manuf Syst 77:356–367. https://doi.org/10.1016/j.jmsy.2024.09.011
52. Wea Song (2023) Flexible job-shop scheduling via graph neural network and deep reinforcement learning. IEEE Trans Ind Inf 19:1600–1610. https://doi.org/10.1109/tii.2022.3189725

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.